

1Voucher — Technical Docs

Operator reference for the platform's value model, contracts, functions and the on-chain events that drive reporting. For a high-level product overview, see the **About** page.

Concepts Payments Gas fees Merchant Contracts Functions Events & Reporting

API

A separate app, not a separate service

`/merchant` is its own React/Vite app (port `5175`, alongside consumer `5173` and this admin console `5174`) — but it talks to the **same Express backend on port 3001**. There is no separate merchant service or database; it's three new route files mounted onto the existing server (`memberAuth.routes.ts`, `settlement.routes.ts`, `merchantSelf.routes.ts`). Run all four with the root `npm run dev` (or `npm run dev:merchant` for just server+merchant).

Why a separate app

A merchant is a business with staff turnover and shared tills — a different trust context from an individual consumer. Wallet-connect (RainbowKit/MetaMask) doesn't fit day-to-day operators, so the merchant app has **no wallet, no wagmi, no RainbowKit** at all. Instead: custody (one corporate account, unchanged from before) is split from access (many operators, each with their own passkey login and a role) and from execution authority (only settlement above a threshold needs a second approval). See **Concepts** for the platform-wide value model this sits alongside.

Org / member model

A merchant (`merchants` row, unchanged) can have many **operators** (`merchant_members`), each scoped to that one merchant with a role:

Role	Can do
cashier	Ring up Point of Sale, view Sales
store_manager	Same as cashier (no extra grants yet — reserved for future store-scoped limits)
org_admin (head office)	Everything above, plus: invite/manage the Team, edit My Business (name/logo/icon/settlement), approve settlement requests above the threshold

Operators sign in with a **passkey** (WebAuthn) — the exact same mechanism as consumer login, but this passkey does *not* back a Safe wallet; it's purely a login credential bound to a `merchant_members` row. First login is a "claim" ceremony: an `org_admin` invites by email/role (`POST /api/member-`